

Reporte de Práctica 01: Paradigmas de la Programación

Estudiante: Carlos Alberto Jaime Mascareño **Matrícula:** 377175 **Sede:** Ensenada, B.C. **Docente:** M.I. José Carlos Gallegos Mariscal

1. Introducción

Este reporte detalla el desarrollo de un sistema de gestión de impresión basado en una estructura de datos de tipo **Cola (FIFO)**. El objetivo es demostrar la transición de un manejo de memoria estática hacia uno dinámico mediante listas enlazadas, analizando las implicaciones en la eficiencia y administración de recursos del sistema.

2. Implementación del Sistema

El programa se divide en módulos para separar la lógica de la cola de la interfaz de usuario. Las funciones implementadas son:

- **enqueue():** Reserva memoria para un nuevo nodo, recibe los datos y lo coloca al final de la cola.
- **dequeue():** Elimina el elemento al frente de la cola y libera su memoria.
- **peek():** Permite observar los datos del próximo trabajo a imprimir sin eliminarlo de la cola.
- **listar():** Recorre la lista completa y muestra todos los trabajos pendientes.
- **simular():** Procesa automáticamente todos los trabajos en orden de llegada hasta vaciar la cola.

3. Demostración de Conceptos Obligatorios

Alcance y Duración de Variables

- **Variables Globales:** Los punteros **frente** y **final** son globales. Su duración es permanente durante toda la vida del programa, permitiendo que todas las funciones manipulen la misma cola.
- **Variables Locales:** Variables como **op** en **main** o **temp** en las funciones son locales; solo existen mientras la función está activa en el Stack.

Manejo de Memoria

- **Heap (malloc):** En la función **enqueue**, se utiliza **malloc(sizeof(Job_t))** para reservar memoria dinámica. Esto permite que la cola crezca según sea necesario.
- **Liberación (free):** En **dequeue**, se utiliza **free(temp)** para liberar el espacio de memoria del nodo procesado, evitando fugas de memoria.

Subprogramas y Punteros

Se utilizan punteros en las funciones porque la cola se modifica directamente. Al pasar o usar punteros globales, los cambios realizados (como mover el **frente**) persisten fuera de la función. El uso de punteros es lo que permite conectar un nodo con el siguiente.

Tipos de Datos: Struct

Se justifica el uso de `struct` para agrupar datos de distintos tipos (enteros para el ID y páginas, arreglos de caracteres para el usuario) en una sola unidad lógica que representa un "Trabajo de Impresión".

4. Explicación de la Simulación

La simulación procesa los trabajos en orden **FIFO** (el primero en entrar es el primero en salir). El programa utiliza un bucle `while` que recorre la cola desde el puntero `frente`. En cada iteración se identifica el trabajo actual, se muestra el progreso de impresión y se llama a `dequeue` para eliminar el trabajo y avanzar al siguiente nodo.

5. Análisis Comparativo: Memoria Estática vs. Dinámica

Cola con Memoria Estática

- **Límites:** El tamaño está definido desde la compilación por un arreglo fijo (ej. `MAX_JOBS = 10`).
- **Eficiencia:** Es ineficiente para el borrado. Al sacar un elemento del frente, se deben mover todos los demás elementos del arreglo una posición a la izquierda (operación shift), lo que consume tiempo $O(n)$.
- **Riesgos:** Desperdicia memoria si el arreglo es muy grande y se usa poco.

Cola con Memoria Dinámica

- **Flexibilidad:** No tiene un límite de tamaño predefinido; crece según el uso de la memoria RAM disponible.
- **Eficiencia:** El proceso de extraer un elemento es instantáneo $O(1)$, ya que solo se actualiza el puntero del frente al siguiente nodo.
- **Complejidad:** Requiere una gestión cuidadosa de punteros para evitar errores de acceso o memoria sin liberar.

6. Conclusiones

Esta práctica demostró que la memoria dinámica es la opción más eficiente para sistemas de impresión reales. Se aprendió a utilizar `malloc` y `free` para gestionar el ciclo de vida de los datos y se reforzó la importancia de las listas enlazadas para optimizar la velocidad de procesamiento en comparación con los arreglos estáticos.

7. Referencias

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.